Architecting Dynamic, Multi-Tenant Feign Clients in Spring Cloud: A Step-by-Step Implementation Guide

Introduction

The evolution of microservice and multi-tenant software-as-a-service (SaaS) architectures has introduced a common yet critical challenge: the need for context-aware routing of outbound API calls. The task of modifying an application to accommodate a new API URL structure, such as changing from a static endpoint to one containing a dynamic placeholder like https://{allianceCode}.localhost:8080, transcends a simple configuration update. It represents a fundamental shift towards a more dynamic, request-aware system. This requirement is a hallmark of sophisticated applications that must serve multiple tenants, route traffic based on user region, or support complex deployment strategies like A/B testing.
The allianceCode in this scenario is a classic tenant identifier, a piece of information specific to an individual incoming request that must dictate the destination of a subsequent outbound API call made via a Spring Cloud OpenFeign client. This architectural pivot necessitates a robust solution that addresses several key engineering questions:

1. What is the most architecturally sound and maintainable pattern for implementing per-request dynamic URL resolution within the Spring Cloud OpenFeign framework?
2. How can the request-specific allianceCode be safely and reliably propagated from the initial point of entry in the application to the component responsible for modifying the outbound request URL, especially in a concurrent, multi-threaded environment?
3. How can this solution be designed to be resilient, comprehensively testable, and free of performance bottlenecks or subtle thread-safety issues that can emerge under load?

This report provides an exhaustive, expert-level guide to architecting and implementing such a solution. It will proceed from foundational concepts of Feign client configuration to a detailed comparative analysis of different architectural strategies. Subsequently, it will offer a prescriptive, step-by-step implementation of the recommended approach, followed by a crucial discussion on production-hardening techniques, including advanced topics like context propagation across asynchronous boundaries and circuit breakers. The final deliverable is a complete blueprint for building a dynamic, multi-tenant Feign client that is not only functional but also scalable, maintainable, and aligned with modern software architecture principles.

Section 1: Foundational Patterns for Feign Client URL Configuration

Before addressing the complexities of per-request dynamic URLs, it is essential to establish a firm understanding of the standard mechanisms for configuring Feign client targets in Spring Cloud. These foundational patterns provide the building blocks upon which a more advanced solution is constructed. The framework's evolution has refined these mechanisms, and a clear

grasp of their intended use is critical for making correct architectural decisions.

1.1 The Duality of @FeignClient: name vs. url

The @FeignClient annotation is the primary mechanism for declaring a declarative REST client. It features two principal attributes for identifying the target service: name and url. Understanding their distinct roles and interplay is fundamental.

The name attribute serves as a logical, arbitrary client name. Its primary function within the Spring Cloud ecosystem is to integrate with service discovery and client-side load balancing. When a service discovery client like Eureka or Consul is on the classpath, the value of the name attribute is used as the service ID to look up available instances of the target service. The framework then uses a Spring Cloud LoadBalancer client to distribute requests among these instances.

The url attribute, in contrast, is used for direct addressing. It allows for the specification of an absolute URL (e.g., https://api.example.com) or a hostname that will be used as the base for all requests made by the client. This approach bypasses service discovery and is suitable for calling external, third-party APIs or services within a known, fixed network location. The user's original implementation, with a static URL provided by Datapower, likely relied on this attribute.

A significant evolution in the framework is that recent versions of Spring Cloud OpenFeign mandate the presence of the name attribute, even when the url attribute is also specified. This is not merely a syntactic requirement; it reflects a deeper architectural principle. By requiring a name, the framework solidifies the concept of each Feign client interface as a distinct, named bean ensemble within the Spring ApplicationContext. As the documentation reveals, Spring creates a new, child ApplicationContext on demand for each named client. It is this per-client context that enables fine-grained, targeted configuration overrides—such as applying a specific RequestInterceptor or ErrorDecoder to one client without affecting others. This mechanism is the very foundation that allows for a clean implementation of the dynamic URL solution detailed later in this report.

1.2 Best Practices in URL Externalization

Hardcoding URLs directly within the @FeignClient annotation is a poor practice, as it tightly couples the application code to a specific environment's configuration. The correct approach is to externalize this configuration, allowing the same application artifact to be deployed across different environments (development, staging, production) without code changes. Spring Boot provides powerful and flexible mechanisms for this.

The most common and recommended method is to define the URL in an external properties file, such as application.yml or application.properties. Spring Cloud OpenFeign provides a standardized property key for this purpose:
spring.cloud.openfeign.client.config.<feignName>.url, where <feignName> corresponds to the name attribute of the @FeignClient annotation.

For the merchant-portal-external-worker, the configuration in application.yml would look as follows:

```
spring:
  cloud:
    openfeign:
      client:
        config:
```

```
          # 'merchant-portal' matches the name attribute in
@FeignClient
          merchant-portal:
            url: https://{allianceCode}.api.example.com
```

The Feign client interface would then omit the url attribute, relying on the external configuration:

```
@FeignClient(name = "merchant-portal")
public interface MerchantPortalClient {
    //... client methods
}
```

An alternative, slightly older pattern involves using property placeholders directly in the annotation's url attribute.

```
@FeignClient(
    name = "merchant-portal",
    url = "${merchant-portal.api.base-url}"
)
public interface MerchantPortalClient {
    //... client methods
}
```

This would be accompanied by a corresponding entry in application.properties:

```
merchant-portal.api.base-url=https://{allianceCode}.api.example.com
```

Both approaches achieve externalization, but the former (spring.cloud.openfeign.client.config...) is generally preferred as it aligns with the framework's structured configuration model for all Feign client properties (e.g., timeouts, logging levels).

1.3 The Limits of Static Placeholders for Per-Request Dynamism

A crucial point of clarification is necessary regarding the nature of property placeholders like ${...}. These placeholders are resolved by the Spring Environment abstraction during the application startup phase, specifically when the bean definitions are being processed and the application context is being refreshed.

This means that a value like ${merchant-portal.api.base-url} is resolved *once* at startup. This mechanism is perfectly suited for managing environment-specific configurations—for example, pointing to a mock server in a dev profile and a live server in a prod profile. However, it is fundamentally incapable of handling values that must change on a per-request basis. The allianceCode is a runtime variable, unique to each incoming transaction. A statically resolved property cannot accommodate this dynamism.

Therefore, while externalizing the URL template https://{allianceCode}.api.example.com is a good first step, a different, runtime-based mechanism is required to substitute the {allianceCode} placeholder with the correct value for each individual API call. This understanding definitively rules out simple property-based solutions and necessitates the exploration of more advanced, programmatic strategies.

Section 2: A Comparative Analysis of Dynamic URL

Strategies

To address the requirement of a per-request dynamic URL, Spring Cloud OpenFeign offers several architectural patterns. Each has distinct trade-offs in terms of code intrusiveness, scalability, and maintainability. A critical evaluation of these strategies is essential to select the most appropriate and robust solution.

2.1 Strategy A: The Method-Argument URI — Direct but Invasive

This strategy involves modifying the signature of each method within the Feign client interface to accept a java.net.URI object as its first, unannotated parameter. The Feign framework has a special provision to interpret this specific parameter as the base URL for that single request, completely overriding any URL defined in the @FeignClient annotation or in external configuration properties.
An implementation for the MerchantPortalClient would look like this:

```
@FeignClient(name = "merchant-portal", url =
"https://placeholder-for-validation.com")
public interface MerchantPortalClient {

    @PostMapping("/v1/transactions")
    ApiResponse createTransaction(URI baseUrl, @RequestBody
TransactionRequest payload);

    @GetMapping("/v1/merchants/{merchantId}")
    MerchantDetails getMerchant(URI baseUrl,
@PathVariable("merchantId") String merchantId);
}
```

To invoke this client, the calling service layer code must now be responsible for constructing the complete base URI for every call:

```
@Service
public class BoardingService {

    @Autowired
    private MerchantPortalClient merchantPortalClient;

    public ApiResponse performBoarding(TransactionRequest payload,
String allianceCode) {
        // Business logic is now polluted with URL construction
details.
        URI dynamicBaseUrl =
URI.create("https://%s.api.example.com".formatted(allianceCode));
        return merchantPortalClient.createTransaction(dynamicBaseUrl,
payload);
    }
}
```

Analysis: While functionally effective and explicit, this pattern is architecturally flawed for this use case. It tightly couples the business logic (the BoardingService) with an infrastructure concern (URL construction). This leads to significant code duplication, as every service method that calls the Feign client must repeat the URI creation logic. It violates the Single Responsibility Principle, making the code harder to maintain, test, and reason about. The responsibility of determining the target URL should be encapsulated and hidden from the client's consumers.

2.2 Strategy B: The RequestInterceptor — Centralized, Non-Invasive, and Recommended

This pattern leverages the feign.RequestInterceptor interface. An interceptor is a component that programmatically intercepts every Feign request *before* it is dispatched, providing an opportunity to modify the request headers, body, or, critically, the target URL.
The implementation involves creating a dedicated interceptor bean that contains the logic for resolving the dynamic URL. This interceptor retrieves the per-request context (the allianceCode), constructs the appropriate URL, and applies it to the outgoing RequestTemplate.
An example interceptor might look like this:

```
public class AllianceUrlRequestInterceptor implements
RequestInterceptor {

    @Override
    public void apply(RequestTemplate template) {
        // Logic to retrieve allianceCode from a request-scoped
context
        String allianceCode = AllianceContextHolder.getAllianceCode();

        if (allianceCode!= null) {
            // Logic to build the new URL and set it on the template
            String resolvedUrl = buildUrlForAlliance(template,
allianceCode);
            template.target(resolvedUrl);
        }
    }
    //... helper methods
}
```

Analysis: This is the most architecturally sound and recommended approach. It perfectly encapsulates the dynamic routing logic within a single, dedicated component. The Feign client interface remains clean, and the services that invoke it are completely unaware of the underlying URL manipulation. This provides excellent separation of concerns, adhering to principles of Aspect-Oriented Programming (AOP). The logic is centralized, making it easy to maintain and evolve. This strategy is non-invasive and scales elegantly as the number of client methods or calling services grows.

2.3 Strategy C: The Multi-Bean contextId — A Niche for Fixed Tenancy

For scenarios with a small, fixed, and predetermined set of tenants, Spring Cloud OpenFeign

allows for the creation of multiple beans of the same Feign client interface, each with a different configuration. To prevent bean name collisions in the Spring container, the contextId attribute of the @FeignClient annotation is used to give each bean a unique identifier.
This would involve defining multiple configurations and clients:

```
// Configuration for Alliance A
public class FeignConfigAllianceA {
    @Bean
    public RequestInterceptor interceptor() { /* adds static headers
for A */ }
}

// Configuration for Alliance B
public class FeignConfigAllianceB {
    @Bean
    public RequestInterceptor interceptor() { /* adds static headers
for B */ }
}

// Two distinct client beans for the same interface
@FeignClient(name="mp-a", contextId="allianceA",
url="https://alliance-a.api.example.com",
configuration=FeignConfigAllianceA.class)
public interface MerchantPortalClientForA extends MerchantPortalClient
{}

@FeignClient(name="mp-b", contextId="allianceB",
url="https://alliance-b.api.example.com",
configuration=FeignConfigAllianceB.class)
public interface MerchantPortalClientForB extends MerchantPortalClient
{}
```

A factory or service would then be needed to select the correct client bean at runtime.
Analysis: This pattern is powerful for managing distinct, static configurations for a known set of clients. However, it is entirely unsuitable for the user's use case. The allianceCode is described as dynamic and specific to each request, implying a potentially large and unbounded set of tenants. Creating a separate bean definition for every possible alliance is not scalable, manageable, or practical. This strategy is included for the sake of completeness and to explicitly highlight why it is the incorrect choice for solving this particular problem.

2.4 Table 1: Architectural Trade-Offs of Dynamic URL Strategies

The following table provides a concise summary of the analysis, comparing the three strategies across key architectural dimensions. This visual breakdown makes the rationale for selecting the RequestInterceptor strategy clear and defensible.

Criterion	Strategy A: Method-Argument URI	Strategy B: RequestInterceptor	Strategy C: Multi-Bean contextId
Code Intrusiveness	High. Requires modifying every method signature in the Feign client interface.	Low. No changes to the Feign client interface or the calling business logic.	Very High. Requires creating new configuration classes and client interfaces/beans for each tenant.
Scalability (for dynamic tenants)	Poor. Logic is duplicated across all call sites, leading to maintenance overhead as the system grows.	Excellent. Logic is centralized in one interceptor, which scales to any number of tenants without code changes.	Not Scalable. Designed for a small, fixed number of tenants. Impractical for dynamic or large numbers of tenants.
Maintainability	Low. Changes to URL logic require finding and updating all call sites. High risk of inconsistency.	High. URL resolution logic is in a single, well-defined location, making it easy to update and manage.	Low. Adding a new tenant requires code changes, recompilation, and redeployment.
Separation of Concerns	Poor. Blurs the line between business logic and infrastructure concerns (URL construction).	Excellent. Perfectly separates the cross-cutting concern of dynamic routing from the business logic.	Fair. Encapsulates configuration per tenant but requires significant boilerplate code.
Performance Overhead	Negligible. Involves simple URI object creation per call.	Negligible. Involves an extra method call and context lookup per request. The overhead is minimal.	Moderate. Involves bean lookups from the FeignContext, which may be slightly more expensive than a direct call.
Best Fit Use Case	Quick-and-dirty solutions or when the URL is genuinely a dynamic data parameter of the business operation.	Multi-tenancy, dynamic routing, A/B testing, or any scenario requiring per-request modification of the target URL.	Systems with a small, fixed number of distinct client configurations (e.g., internal vs. external API versions).

As the table demonstrates, the RequestInterceptor strategy is superior across all critical dimensions for the specified problem: it is non-invasive, highly scalable, maintainable, and provides a clean separation of concerns. It is the unequivocally recommended approach.

Section 3: A Step-by-Step Implementation of the RequestInterceptor Solution

This section provides a prescriptive, code-centric guide to implementing the recommended RequestInterceptor architecture. The process is broken down into four distinct, logical steps,

forming a clear path from capturing the request context to applying the dynamic URL.

3.1 Step 1: Establishing the Alliance Context with a Servlet Filter

The first requirement is to capture the allianceCode at the earliest point in the request lifecycle. The allianceCode is part of the incoming request's context, likely transmitted as an HTTP header (e.g., X-Alliance-Code) or embedded within a JWT. A standard jakarta.servlet.Filter is the ideal component for this task, as it executes before the request reaches the Spring MVC DispatcherServlet and the application's controllers.
This filter will be responsible for extracting the allianceCode and storing it in a location accessible to downstream components.
Implementation:

```
package com.example.boarding.worker.filter;

import com.example.boarding.worker.context.AllianceContextHolder;
import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.stereotype.Component;
import java.io.IOException;

@Component
public class AllianceContextFilter implements Filter {

    private static final String ALLIANCE_CODE_HEADER =
"X-Alliance-Code";

    @Override
    public void doFilter(ServletRequest request, ServletResponse
response, FilterChain chain)
            throws IOException, ServletException {
        HttpServletRequest httpServletRequest = (HttpServletRequest)
request;
        String allianceCode =
httpServletRequest.getHeader(ALLIANCE_CODE_HEADER);

        try {
            // Set the alliance code for the current thread's context.
            AllianceContextHolder.setAllianceCode(allianceCode);
            chain.doFilter(request, response);
        } finally {
            // CRITICAL: Always clear the context after the request is
processed.
            AllianceContextHolder.clear();
        }
```

```
    }
}
```

This filter is registered as a Spring @Component to be automatically picked up and added to the servlet filter chain. The try...finally block is of paramount importance: it guarantees that the context is cleared after the request has been fully processed, preventing context from one request from leaking into a subsequent request handled by the same recycled thread from the web server's thread pool.

3.2 Step 2: Propagating Context via a ThreadLocal Holder

The RequestInterceptor that will modify the URL is typically a singleton bean, meaning it cannot safely store request-specific state (like the allianceCode) in its instance variables. Doing so would create severe race conditions in a multi-threaded environment. The standard solution to this problem in imperative Java applications is to use a ThreadLocal variable. This mechanism stores data that is privately scoped to the current execution thread.
To provide a clean, type-safe API, it is best practice to wrap the ThreadLocal in a dedicated holder class.
Implementation:

```
package com.example.boarding.worker.context;

public final class AllianceContextHolder {

    // Using InheritableThreadLocal allows context to be passed to
child threads spawned from the request thread.
    // While not strictly necessary for the basic interceptor case,
it's a good practice for more complex async scenarios.
    private static final ThreadLocal<String> CONTEXT = new
InheritableThreadLocal<>();

    private AllianceContextHolder() {
        // Private constructor to prevent instantiation
    }

    public static void setAllianceCode(String allianceCode) {
        if (allianceCode!= null) {
            CONTEXT.set(allianceCode);
        } else {
            // Ensure that setting a null value effectively clears it.
            CONTEXT.remove();
        }
    }

    public static String getAllianceCode() {
        return CONTEXT.get();
    }

    public static void clear() {
```

```
        CONTEXT.remove();
    }
}
```

This static utility class provides a clear and safe interface (set, get, clear) for managing the allianceCode associated with the current request thread. The AllianceContextFilter from Step 1 uses this class to store and clear the context.

3.3 Step 3: Constructing the Dynamic URL Interceptor

With the context propagation mechanism in place, the core component—the RequestInterceptor—can now be built. Its sole responsibility is to retrieve the allianceCode from the AllianceContextHolder and manipulate the RequestTemplate to point to the correct tenant-specific URL.
A common pitfall here is to use simple string replacement on the URL. A more robust and safer method is to use a proper URI-building tool like Spring's UriComponentsBuilder to handle URI syntax, encoding, and assembly correctly. However, for this specific use case of replacing a host-level placeholder, direct manipulation of the target is also possible and often simpler. The key is to access the *unresolved* URL template from the Feign client definition, not the partially processed request path. This can be accessed via template.feignTarget().url().
Implementation:

```
package com.example.boarding.worker.feign;

import com.example.boarding.worker.context.AllianceContextHolder;
import feign.RequestInterceptor;
import feign.RequestTemplate;
import org.springframework.util.StringUtils;

public class AllianceUrlRequestInterceptor implements
RequestInterceptor {

    @Override
    public void apply(RequestTemplate template) {
        String allianceCode = AllianceContextHolder.getAllianceCode();

        // Fail-fast if the required context is missing.
        if (!StringUtils.hasText(allianceCode)) {
            throw new IllegalStateException("AllianceCode is missing
from the request context. Cannot determine target URL.");
        }

        // Get the original URL template from the @FeignClient
annotation (e.g., "https://{allianceCode}.api.example.com").
        // This is more reliable than trying to parse template.url()
which might already be partially resolved.
        String urlTemplate = template.feignTarget().url();

        // Perform the replacement to create the final, resolved base
```

```
URL.
        String resolvedUrl = urlTemplate.replace("{allianceCode}",
allianceCode);

        // Set the final base URL on the request template. Feign will
append the method-specific path to this.
        template.target(resolvedUrl);
    }
}
```

This interceptor is designed to be stateless. It retrieves all necessary information from the ThreadLocal context and the RequestTemplate passed into the apply method. By throwing an IllegalStateException, it ensures that misconfigured requests fail loudly and clearly. The use of template.target() correctly informs Feign of the new base URL for the request.

3.4 Step 4: Targeted Feign Client Configuration

The final step is to apply the AllianceUrlRequestInterceptor exclusively to the MerchantPortalClient. Applying it globally to all Feign clients in the application would be incorrect and could cause errors for clients that do not require this dynamic URL logic. Spring Cloud OpenFeign enables this targeted configuration via the configuration attribute on the @FeignClient annotation. A configuration class is created to provide the interceptor as a @Bean. Crucially, this configuration class must *not* be a top-level @Configuration or @Component, otherwise Spring's component scanning would register the interceptor bean globally, defeating the purpose of targeted application.

Implementation:

1. **Create the Per-Client Configuration Class:**

```
package com.example.boarding.worker.feign;

import feign.RequestInterceptor;
import org.springframework.context.annotation.Bean;

// IMPORTANT: This class is NOT annotated with @Configuration.
// It is a plain class that will be explicitly imported by the
Feign client.
public class MerchantPortalFeignConfig {

    @Bean
    public RequestInterceptor allianceUrlRequestInterceptor() {
        return new AllianceUrlRequestInterceptor();
    }
}
```

2. **Update the Feign Client Interface:**Modify the MerchantPortalClient to reference the configuration class and use the placeholder in its url attribute.

```
package com.example.boarding.worker.feign;

import org.springframework.cloud.openfeign.FeignClient;
```

```
// other imports

@FeignClient(
    name = "merchant-portal",
    url = "https://{allianceCode}.api.example.com", // The URL
template with the placeholder
    configuration = MerchantPortalFeignConfig.class // Apply the
specific configuration
)
public interface MerchantPortalClient {

    @PostMapping("/api/v1/onboard")
    OnboardingResponse onboardMerchant(@RequestBody
OnboardingRequest request);

    //... other methods
}
```

This completes the implementation. The logical flow is now fully established: an incoming request passes through the AllianceContextFilter, which populates the AllianceContextHolder. When a method on MerchantPortalClient is called, the AllianceUrlRequestInterceptor (provided by the client-specific MerchantPortalFeignConfig) is invoked. It reads the allianceCode from the context holder and dynamically constructs and sets the correct target URL for the outbound request, all without the business logic being aware of the process.

Section 4: Production-Hardening and Advanced Considerations

Implementing the functional solution is only part of the task. A production-grade system must also be resilient, thread-safe under load, and robust in the face of asynchronous execution and errors. This section addresses these critical, advanced considerations.

4.1 Concurrency and Thread-Safety Guarantees

A primary concern in any server-side application is thread safety. The Feign framework itself is designed with this in mind. Feign client instances, once created, are thread-safe and can be shared across multiple threads. However, this guarantee extends only to the core framework; any custom components plugged into Feign, such as encoders, decoders, or our RequestInterceptor, must also be thread-safe.
The architecture presented in this report is inherently thread-safe due to two key design choices:

1. **Stateless Interceptor:** The AllianceUrlRequestInterceptor is a stateless singleton. It does not contain any instance variables that hold request-specific data. All the information it needs is either passed into the apply method via the RequestTemplate or retrieved from a thread-safe source.
2. **ThreadLocal Context:** The use of AllianceContextHolder confines the request-specific state (the allianceCode) to the individual thread processing that request. This prevents

any possibility of data from one request overwriting or being read by another concurrent request, thus eliminating race conditions.

This combination of a stateless component with thread-scoped state is a standard and robust pattern for safely handling request context in concurrent Java applications.

4.2 Navigating Circuit Breakers and Asynchronous Execution

The most complex challenge in context propagation arises when asynchronous processing is introduced. A common use case is protecting Feign clients with a circuit breaker, such as Resilience4J (the modern replacement for Hystrix). If the circuit breaker is configured with a THREAD isolation strategy, it executes the Feign call on a separate, dedicated thread pool. This "thread-hopping" poses a significant problem: the InheritableThreadLocal context established on the initial web server thread is **lost** when execution jumps to the circuit breaker's thread pool. Consequently, AllianceContextHolder.getAllianceCode() will return null inside the interceptor, causing the request to fail.

There are several ways to address this, with a clear evolution from older, more manual methods to modern, automated solutions.

1. **The SEMAPHORE Isolation Strategy (Legacy Approach):** An older solution, particularly with Hystrix, was to change the isolation strategy from THREAD to SEMAPHORE. Semaphore isolation executes the Feign call on the original calling thread, avoiding the thread-hop and thus preserving the ThreadLocal context. While this solves the context propagation problem, it comes at a cost: it defeats the purpose of bulkhead isolation that thread pools provide and can limit the system's throughput and resilience under certain failure modes.
2. **The Micrometer Context Propagation Library (Modern, Recommended Approach):** For modern Spring Boot 3 applications, the definitive solution is to leverage the Micrometer Context Propagation library. This library is designed specifically to solve the problem of propagating context across thread boundaries in a standardized way.When dependencies like micrometer-tracing-bridge-brave or micrometer-tracing-bridge-otel are included in a project, Spring Boot's auto-configuration instruments key components like TaskExecutors. This instrumentation automatically wraps tasks submitted to thread pools, ensuring that ThreadLocal values (along with tracing information like trace IDs and spans) are captured from the parent thread and restored on the execution thread.To ensure this works with Resilience4J's thread pools, you simply need the correct dependencies on the classpath. The framework handles the propagation automatically, making the ThreadLocal approach work seamlessly even with THREAD isolation. This is a vastly superior solution as it is non-invasive, declarative, and aligns with modern observability practices, solving not just for the allianceCode but for all thread-scoped context.

4.3 Robust Error Handling and Fallback Logic

A production system must gracefully handle failures. In the context of the MerchantPortalClient, this involves two primary scenarios.

First, if the allianceCode is missing from the request context, the interceptor correctly throws an IllegalStateException. This is a "fail-fast" approach that prevents an invalid request from being sent. This exception can be handled globally by a Spring @ControllerAdvice to return a 400 Bad Request status to the original caller, indicating a client-side error.

Second, the remote API call itself may fail due to network issues, server errors, or timeouts.

Spring Cloud OpenFeign provides robust mechanisms for handling these failures. A custom feign.ErrorDecoder can be implemented to translate HTTP error statuses into specific, checked exceptions. Alternatively, if a circuit breaker is in use, a fallback method or class can be specified in the @FeignClient annotation. This fallback provides an alternative execution path when the primary call fails or the circuit is open, allowing the system to return a default response, a cached value, or a standardized error message, thereby improving the system's overall resilience. This error handling logic should be defined as a bean within the client-specific MerchantPortalFeignConfig to ensure it applies only to the MerchantPortalClient.

4.4 A Blueprint for Testing

Thorough testing is non-negotiable. The dynamic routing logic must be validated at both the unit and integration levels.

Unit Testing the Interceptor: The AllianceUrlRequestInterceptor can be tested in isolation as a plain Java object. A JUnit test can verify its logic directly.

```
@Test
void apply_shouldSetCorrectTargetUrl_whenAllianceCodeIsPresent() {
    // Arrange
    AllianceUrlRequestInterceptor interceptor = new
AllianceUrlRequestInterceptor();
    RequestTemplate template = new RequestTemplate();
    template.feignTarget(new Target.HardCodedTarget<>(
        MerchantPortalClient.class,
        "merchant-portal",
        "https://{allianceCode}.api.example.com"
    ));
    AllianceContextHolder.setAllianceCode("acme-corp");

    // Act
    interceptor.apply(template);

    // Assert
    // Note: template.url() is now the path, the full URL is part of
the final Request.
    // The most reliable assertion is on the result of
target.apply(template).
    Request request = new Target.Resolved<>(interceptor,
template.feignTarget()).apply(template);

assertThat(request.url()).startsWith("https://acme-corp.api.example.co
m");

    // Cleanup
    AllianceContextHolder.clear();
}

@Test
void apply_shouldThrowException_whenAllianceCodeIsMissing() {
```

```
    // Arrange
    AllianceUrlRequestInterceptor interceptor = new
AllianceUrlRequestInterceptor();
    RequestTemplate template = new RequestTemplate();
    template.feignTarget(new Target.HardCodedTarget<>(
        MerchantPortalClient.class,
        "merchant-portal",
        "https://{allianceCode}.api.example.com"
    ));

    // Act & Assert
    assertThrows(IllegalStateException.class, () ->
interceptor.apply(template));
}
```

Integration Testing with WireMock: The most crucial test is an end-to-end integration test that verifies the entire flow. WireMock is the ideal tool for this, allowing the creation of a mock HTTP server that can simulate the external Merchant Portal API.
The test would proceed as follows:

1. Set up a Spring Boot integration test using @SpringBootTest and a WireMock server instance managed by a JUnit 5 extension or rule.
2. In the test setup, configure WireMock to expect requests to multiple, distinct tenant URLs. For example, stub a POST request to /api/v1/onboard on https://alliance-a.api.example.com to return a success response, and another for https://alliance-b.api.example.com.
3. In the test method, invoke the application's service layer twice in succession. Before the first call, set the context with AllianceContextHolder.setAllianceCode("alliance-a"). Before the second call, set it with AllianceContextHolder.setAllianceCode("alliance-b").
4. After the calls, use WireMock's verify() methods to assert that one request was received at the alliance-a URL and another was received at the alliance-b URL.

This integration test provides definitive proof that the entire mechanism—from the servlet filter through the ThreadLocal context to the interceptor's dynamic URL resolution—is working correctly as a cohesive unit.

Conclusion and Implementation Checklist

The challenge of adapting a Feign client to a dynamic, tenant-specific URL structure is a sophisticated engineering task that requires a careful architectural approach. A simple property update is insufficient; a runtime solution is necessary. Through a comparative analysis of available patterns, this report has established that implementing a custom feign.RequestInterceptor is the most robust, maintainable, and scalable solution.
The recommended architecture—a non-invasive RequestInterceptor that reads tenant context from a ThreadLocal, which is populated by an upstream Filter—provides a clean separation of concerns. It isolates the infrastructure logic of dynamic routing from the business logic, requires no changes to the Feign client's method signatures, and scales to any number of tenants. When hardened with modern context propagation libraries like Micrometer Context Propagation, this pattern remains resilient even in complex asynchronous scenarios involving circuit breakers with

thread pool isolation.
To ensure a successful and complete implementation, the following checklist provides an actionable summary of the required steps.

Final Implementation Checklist

1. [ ] **Dependency Verification:** Confirm that the spring-cloud-starter-openfeign dependency is present in the pom.xml or build.gradle file of the merchant-portal-external-worker module.
2. [ ] **URL Template Externalization:** In application.yml (or .properties), define the base URL template for the merchant-portal client using the spring.cloud.openfeign.client.config.merchant-portal.url property. The value should be https://{allianceCode}.api.example.com or the equivalent pattern.
3. [ ] **Context Capture Filter:** Implement a jakarta.servlet.Filter (e.g., AllianceContextFilter) annotated with @Component to extract the allianceCode from the incoming HttpServletRequest headers.
4. [ ] **ThreadLocal Context Holder:** Create a final utility class (e.g., AllianceContextHolder) with a private constructor and static set, get, and clear methods wrapping an InheritableThreadLocal.
5. [ ] **Integrate Context Holder:** In the doFilter method of the context filter, wrap the chain.doFilter(...) call in a try...finally block. Call AllianceContextHolder.setAllianceCode(...) in the try block and, most critically, call AllianceContextHolder.clear() in the finally block to prevent context leakage.
6. [ ] **Dynamic URL Interceptor:** Implement the AllianceUrlRequestInterceptor class. Inside the apply method, retrieve the allianceCode from the context holder, get the URL template from template.feignTarget().url(), construct the resolved URL, and set it using template.target(resolvedUrl). Include a null-check for the allianceCode to fail fast.
7. [ ] **Per-Client Feign Configuration:** Create a plain Java class (e.g., MerchantPortalFeignConfig) that is **not** annotated with @Configuration or @Component. Within this class, define a @Bean method that returns a new instance of your AllianceUrlRequestInterceptor.
8. [ ] **Apply Targeted Configuration:** Modify the @FeignClient annotation on the MerchantPortalClient interface. Ensure it has a name attribute, a url attribute pointing to the placeholder template, and the configuration attribute set to MerchantPortalFeignConfig.class.
9. [ ] **Review Asynchronous Boundaries:** If the Feign client is protected by a circuit breaker (like Resilience4J) using THREAD isolation, ensure the micrometer-tracing-bridge-brave or micrometer-tracing-bridge-otel dependency is on the classpath to enable automatic context propagation. If not, consider switching to SEMAPHORE isolation as a temporary measure.
10. [ ] **Comprehensive Testing:** Write a JUnit test for the AllianceUrlRequestInterceptor in isolation. Subsequently, create a full @SpringBootTest integration test using WireMock to simulate the external API and verify that requests are correctly routed to different tenant-specific URLs based on the context set for each call.

Works cited

1. Spring Cloud OpenFeign,

https://docs.spring.io/spring-cloud-openfeign/docs/current/reference/html/ 2. 7. Declarative REST Client: Feign - Spring Cloud Project, https://cloud.spring.io/spring-cloud-netflix/multi/multi_spring-cloud-feign.html 3. Spring Cloud OpenFeign Features, https://docs.spring.io/spring-cloud-openfeign/reference/spring-cloud-openfeign.html 4. Configuring Spring Cloud FeignClient URL | Baeldung, https://www.baeldung.com/spring-cloud-feignclient-url 5. Introduction to Spring Cloud OpenFeign - Baeldung, https://www.baeldung.com/spring-cloud-openfeign 6. Spring Cloud OpenFeign - VMware, https://docs.spring.vmware.com/spring-cloud-openfeign/docs/4.0.7/reference/html/index.html 7. Externalized Configuration :: Spring Boot, https://docs.spring.io/spring-boot/reference/features/external-config.html 8. Spring Feign: support configuration properties placeholders using in @FeignClient attributes values : IDEA-242010 - JetBrains YouTrack, https://youtrack.jetbrains.com/issue/IDEA-242010/Spring-Feign-support-configuration-properties-placeholders-using-in-FeignClient-attributes-values 9. spring cloud - How can I change the feign URL during the runtime? - Stack Overflow, https://stackoverflow.com/questions/43733569/how-can-i-change-the-feign-url-during-the-runtime 10. Unexpected behaviour of a Feign client with Spring MVC URI parameters #895 - GitHub, https://github.com/spring-cloud/spring-cloud-openfeign/issues/895 11. Multiple Feign clients with same name fails application startup when url from configuration properties #960 - GitHub, https://github.com/spring-cloud/spring-cloud-openfeign/issues/960 12. Is Feign threadsafe...? - Stack Overflow, https://stackoverflow.com/questions/38916892/is-feign-threadsafe 13. Microservice architecture: Using Java thread locals and Tomcat/Spring capabilities for automated information propagation - N47, https://www.north-47.com/microservice-architecture-using-java-thread-locals-and-tomcat-spring-capabilities-for-automated-information-propagation/ 14. Spring open feign set custom header using baggage (or trace ..., https://stackoverflow.com/questions/78546263/spring-open-feign-set-custom-header-using-baggage-or-trace 15. Encoding of URI Variables on RestTemplate | Baeldung, https://www.baeldung.com/spring-resttemplate-uri-variables-encode 16. URI Links :: Spring Framework, https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-uri-building.html 17. 通过feign的RequestInterceptor 修改实际调用服务名称原创 - CSDN博客, https://blog.csdn.net/connection_/article/details/128651967 18. How to get the url · Issue #848 · spring-cloud/spring-cloud-openfeign - GitHub, https://github.com/spring-cloud/spring-cloud-openfeign/issues/848 19. How to set a custom Feign RequestInterceptor for specific clients? - Stack Overflow, https://stackoverflow.com/questions/56847094/how-to-set-a-custom-feign-requestinterceptor-for-specific-clients 20. feign is threadsafe? · Issue #875 · OpenFeign/feign - GitHub, https://github.com/OpenFeign/feign/issues/875 21. FEIGN + OAUTH2 calls from another thread not propagating security #1330 - GitHub, https://github.com/spring-cloud/spring-cloud-netflix/issues/1330 22. Copying current request information into Feign interceptor with Hystrix enabled, https://arnoldgalovics.com/feign-interceptor-hystrix/ 23. Spring Batch and Feign OAuth2 RequestInterceptor - JDriven Blog, https://jdriven.com/blog/2016/08/spring-batch-feign-oauth2-requestinterceptor 24. 1. Declarative REST Client: Feign - Spring Cloud Project,

https://cloud.spring.io/spring-cloud-openfeign/multi/multi_spring-cloud-feign.html 25. Context Propagation with Project Reactor 3 - Unified Bridging between Reactive and Imperative - Spring,
https://spring.io/blog/2023/03/30/context-propagation-with-project-reactor-3-unified-bridging-between-reactive/ 26. Observability with Spring Boot 3,
https://spring.io/blog/2022/10/12/observability-with-spring-boot-3/